НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ “ЛЬВІВСЬКА ПОЛІТЕХНІКА”
КАФЕДРА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

Угриновський Богдан

МЕТОДИЧНІ ВКАЗІВКИ
ДО ВИКОНАННЯ ЛАБОРАТОРНОЇ РОБОТИ №3
З ДИСЦИПЛІНИ «КРОСПЛАТФОРМНЕ ПРОГРАМУВАННЯ»
ДЛЯ СТУДЕНТІВ першого (бакалаврського) рівня вищої освіти
СПЕЦІАЛЬНОСТІ 121
«ІНЖЕНЕРІЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ»

Львів - 2026

ЛАБОРАТОРНА РОБОТА №3

Тема. Створення базової структури застосунку та основних екранів відповідно до розробленого дизайну.

Мета. Оволодіти базовими навичками створення графічного інтерфейсу кросплатформного застосунку у Flutter та реалізувати частину основних екранів відповідно до попередньо розробленого дизайну.

ТЕОРЕТИЧНІ ВІДОМОСТІ

Основи побудови UI у Flutter

Flutter – це реактивний декларативний UI-фреймворк. Розробник визначає відображення стану застосунку у вигляді стану інтерфейсу, а фреймворк автоматично оновлює інтерфейс під час зміни даних у застосунку. Ця модель була натхненна ідеями, закладеними у фреймворку React від Facebook, який переосмислив багато традиційних принципів проєктування інтерфейсів.

У більшості традиційних UI-фреймворків початковий стан інтерфейсу задається один раз, а далі його потрібно окремо оновлювати вручну в коді під час роботи програми у відповідь на події. Основна проблема такого підходу полягає в тому, що зі зростанням складності застосунку розробник мусить відстежувати, як зміни стану впливають на всі частини інтерфейсу.

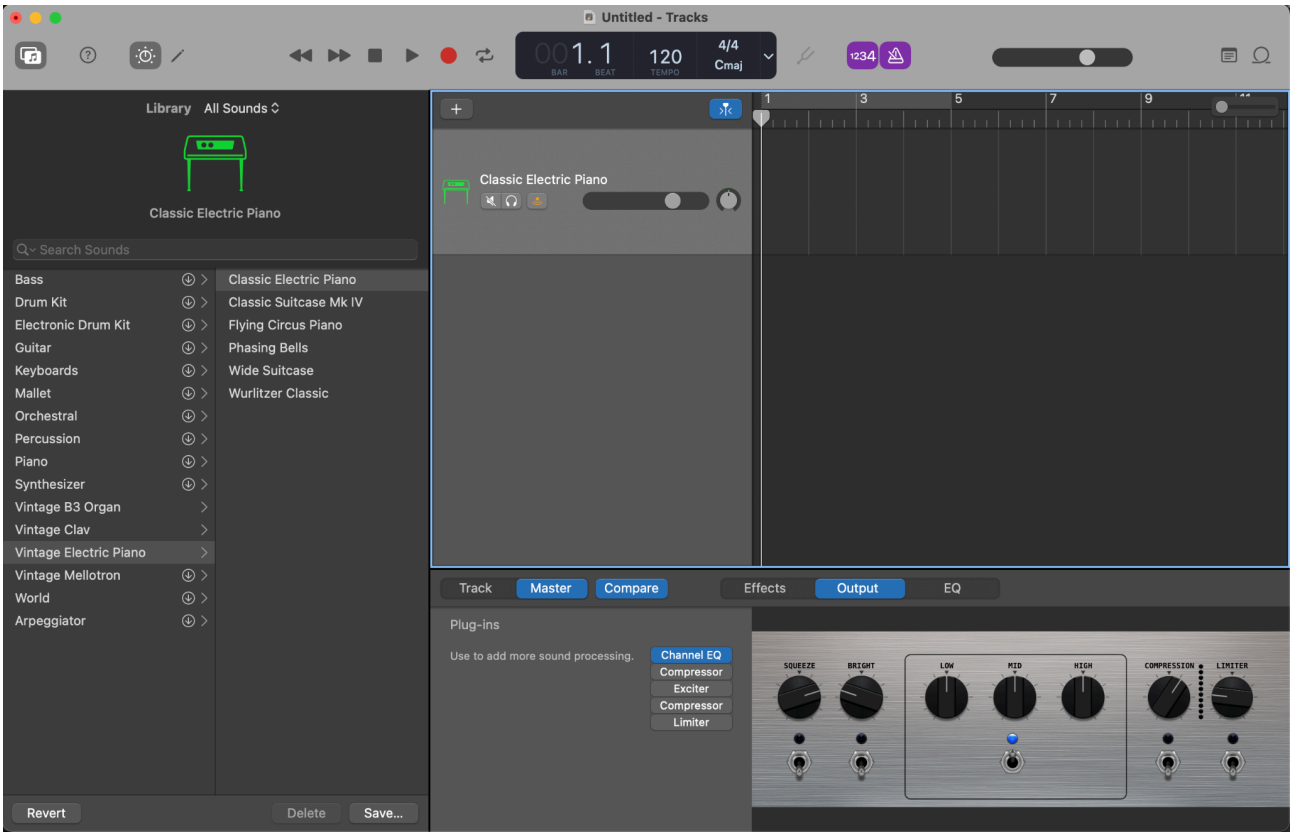


Рис. 1. Приклад графічного інтерфейсу застосунку “GarageBand”.

У користувацькому інтерфейсі (рис. 1) існує багато точок, де може змінюватися стан: кнопки керування програвачем, перемикачі виводу, список звукових ефектів тощо. Під час взаємодії користувача ці зміни мають коректно відображатися у всіх пов’язаних елементах. Проблема в тому, що навіть невелика зміна в одній частині інтерфейсу може спричинити несподівані наслідки в іншому коді.

Одним із класичних рішень вважається підхід MVC: зміни надходять у модель (Model) через контролер (Controller), а модель через нього ж оновлює представлення (View). Проте і тут є труднощі – створення й оновлення елементів UI залишаються окремими кроками, які легко розсинхронізуються.

Flutter та інші реактивні фреймворки застосовують інший підхід – вони явно розділяють інтерфейс і його стан. Завдяки React-подібним API розробник лише описує, як має виглядати UI при певному стані, а сам фреймворк вже відповідає за його створення чи оновлення.

У Flutter інтерфейс будується за допомогою віджетів (аналогів компонентів у React). Віджети – це незмінні класи, що формують дерево

віджетів. Віджет у Flutter описує свій інтерфейс через метод build(), який перетворює стан у UI:

UI = f(state).

Метод *build()* спроектований для швидкого виконання та перебудови цілого дерева віджетів і має бути без побічних ефектів для того, щоб фреймворк міг викликати його скільки завгодно разів (навіть один раз на кожен кадр). Побічні ефекти в даному випадку – це різного роду складні обчислення, читання бази даних чи доступ до віддаленого серверу, які потрібно виносити за межі цього методу під час розробки застосунку. Використання *await* в методі *build()* заборонено, а складні обчислення можуть дуже негативно впливати на затримки виконання методу *build()*.

Такий підхід можливий завдяки характеристикам середовища виконання мови: зокрема, швидкому створенню та знищенню об’єктів. Мова Dart добре підходить для цієї задачі:

- механізм збирача сміття Dart забезпечує ефективність роботи з великою кількістю малих об’єктів, які можуть часто видалятись і створюватись, тому дерева віджетів можуть часто перестворюватись без необхідності їх кешування;
- константні конструктори і застосування const віджетів дозволяє Dart повторно використовувати той самий об’єкт, а Flutter – пропускати зайву роботу з оновлення цього віджета, коли перебудовується батьківський віджет.

Конвеєр рендерингу Flutter

Конвеєр рендирингу Flutter передбачає етапи Build, Layout та Painting, які забезпечується трьома рівнями дерев для перетворення опису інтерфейсу користувача у конкретні об’єкти, які будуть розміщуватись і малюватись на екрані графічним рушієм.

Дерево віджетів у Flutter описує структуру інтерфейсу: які елементи містить екран, як вони вкладені один в одного та як розташовані. Кожен віджет

є вузлом дерева. Батьківський віджет містить дочірні та задає умови їхнього розміщення.

Розгляньмо фрагмент коду, який демонструє ієрархію віджетів:

```
Container(
  color: Colors.green,
  child: Column(
    children: [
      Image.network('https://www.example.com/flutter.png'),
      const Text('Вивчаємо Flutter'),
    ],
  ),
);
```

Коли Flutter потрібно відобразити цей фрагмент, він викликає метод *build()*, який повертає піддерево віджетів (рис. 2), що описує інтерфейс відповідно до поточного стану застосунку. Під час цього процесу метод *build()* за потреби може додавати нові віджети залежно від стану.

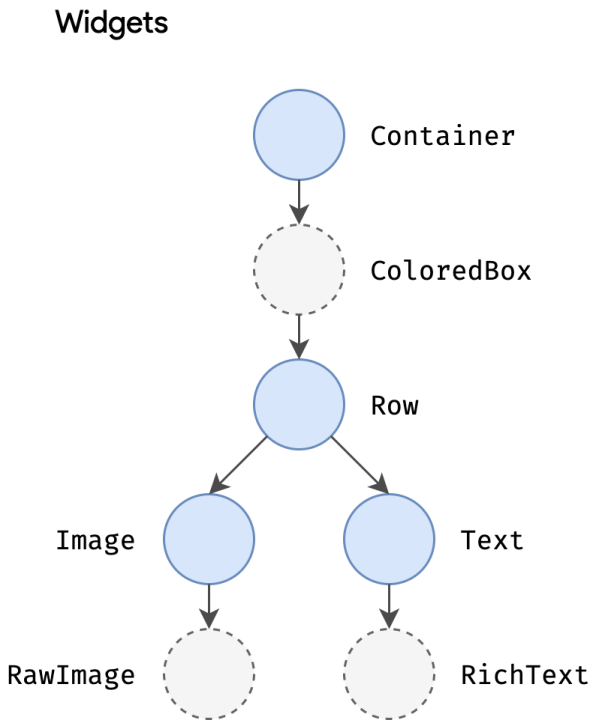


Рис. 2. Приклад дерева віджетів.

Основне призначення дерева віджетів:

- побудова інтерфейсу – поєднання простих віджетів у складні екрани;

- визначення вкладеності та розташування, наприклад, текст усередині кнопки або елементи в колонці;

- оновлення інтерфейсу – після зміни стану Flutter перебудовує відповідні частини дерева та визначає необхідні зміни на екрані;

- доступ до спільних даних – через BuildContext віджет може отримати тему та інші дані від предків.

Дерево віджетів – це декларативний спосіб описати інтерфейс користувача. Для керування життєвим циклом дерева і фактичного розміщення та малювання Flutter також використовує дерево елементів (element tree) і дерево об’єктів рендерингу (render tree), тому у своїй основі Flutter – це набір механізмів, які ефективно обходять змінені частини дерева віджетів, трансформують їх у нижчі рівні представлення дерев та поширюють зміни між цими рівнями (рис. 3).

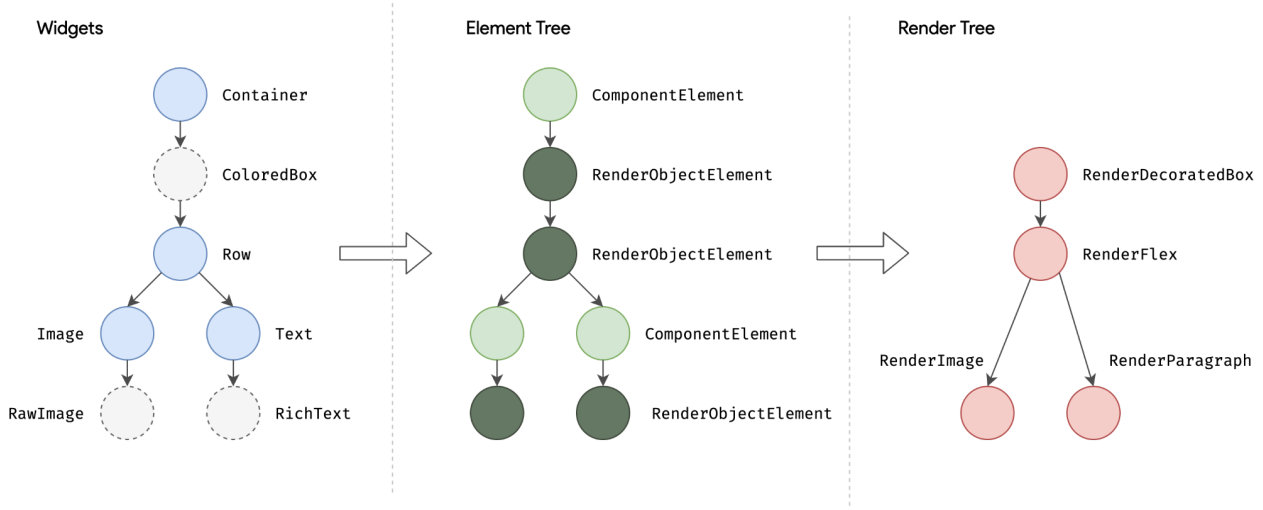


Рис. 3. Обхід та трансформація дерев віджетів, елементів та рендеру.

Дерево елементів у Flutter – це постійна структура, яка представляє віджети, що зараз використовуються в інтерфейсі. Кожен елемент (Element) відповідає певному віджету та визначає його місце в дереві.

Його основні функції:

- Оновлення інтерфейсу. Коли створюється новий опис віджетів, Flutter оновлює наявні елементи, якщо тип і ключ (key) відповідних віджетів збігаються.

- Збереження стану. Для StatefulWidget елемент утримує об’єкт State, тому дані можуть зберігатися між перебудовами.

- Керування життєвим циклом. Елементи додаються в дерево, оновлюються та видаляються.

- Зв’язок із рендерингом. Відповідні елементи створюють і оновлюють об’єкти, які розміщують та малюють інтерфейс.

Також BuildContext – це інтерфейс елемента, через який можна звертатися до предків, наприклад, щоб отримати тему через *Theme.of(context)*.

Дерево об’єктів рендерингу (RenderObject) у Flutter відповідає за розміщення та малювання інтерфейсу. Воно визначає, як описані віджетами елементи виглядатимуть на екрані.

Його основні функції:

- Розрахунок розмірів і розташування (layout). Визначає розміри елементів та їхні позиції з урахуванням обмежень батьківських об’єктів.

- Малювання (paint). Формує команди для відображення тексту, зображень, фону та інших складових.

- Визначення місця взаємодії (hit testing). Встановлює, на який елемент припало натискання або дотик.

Наприклад, *Column* використовує об’єкт *RenderFlex*, який розташовує дочірні елементи вертикально.

Не кожен віджет має власний об’єкт рендерингу, тому це дерево зазвичай менш деталізоване, ніж дерево віджетів.

Основи мови Dart для побудови UI у Flutter

Dart – це об’єктно-орієнтована мова програмування з синтаксисом, подібним до Java, C#, JavaScript. Має JIT (Just-in-Time) компіляцію для швидкого відлагодження та AOT (Ahead-of-Time) компіляцію у машинний код для продуктивних застосунків. У Flutter Dart дозволяє створювати реактивні інтерфейси, де віджети описуються декларативно.

Класи. У Flutter все – віджет, а віджети – це звичайні класи Dart. Можна створювати власні класи для повторного використання інтерфейсних елементів. Ключові особливості: підтримка успадкування, інтерфейси через *abstract class*, перевизначення методів (*@override*).

```
class MyButton extends StatelessWidget {
  final String label;
  const MyButton({required this.label});

  @override
  Widget build(BuildContext context) {
    return ElevatedButton(onPressed: () {}, child: Text(label));
  }
}
```

Конструктори. У Flutter більшість віджетів приймають параметри через конструктори. Використання *const*-конструкторів дозволяє зменшити кількість створюваних об’єктів.

```
const Text(
  'Привіт, Flutter!',
  style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold),
)
```

Параметри. Іменовані параметри (оголошуються у фігурних дужках) часто застосовуються у віджетах. Можуть бути *required* або мати значення за замовчуванням.

```
Container(
  margin: const EdgeInsets.all(10),
  padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
  color: Colors.blue,
  child: const Text("Hello!"),
)
```

Колекції. *List* найчастіше використовується для масивів віджетів (*children: []*). *Map* для роботи з парами ключ-значення, наприклад для локалізації чи темізації. Підтримка collection-if та collection-for дозволяє генерувати віджети умовно чи у циклах.

```
Column(
  children: [
    for (var i = 0; i < 3; i++) Text('Item $i'),
    if (true) const Text('Visible!'),
  ],
)
```

Функції та колбеки. Функції можна передавати як параметри (наприклад, у *onPressed*). Dart підтримує лямбда-функції (arrow functions) для компактного коду.

```
ElevatedButton(
  onPressed: () => print("Button clicked!"),
  child: const Text("Press me"),
```

```
)
```

Null safety. Dart має систему null safety – змінні за замовчуванням не можуть містити *null*. Для позначення можливого *null* використовується *?*. У Flutter це знижує кількість помилок у UI-коді.

```
String? username; // може бути null
username ??= "Guest"; // оператор ??=
```

Іммутабельність та ключове слово const. У Flutter дуже важливо використовувати *const* там, де віджет не змінюється. Це оптимізує роботу фреймворку, адже повторно використовуються існуючі об’єкти, замість створення нових.

```
const Icon(Icons.star, color: Colors.yellow);
```

Асинхронність у UI. У багатьох випадках (завантаження даних, робота з мережею, Firebase) використовуються *Future* та *async/await*. У Flutter асинхронні виклики часто поєднуються з віджетами на кшталт *FutureBuilder* чи *StreamBuilder*.

```
FutureBuilder<String>(
  future: fetchData(),
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting) {
      return const CircularProgressIndicator();
    }
    return Text(snapshot.data ?? "No data");
  },
)
```

Інтеграція з UI через Dart. У Flutter Dart описує стан і перетворює його на UI. Для цього використовуються *StatelessWidget* та *StatefulWidget*. Dart дозволяє ефективно перебудовувати тільки ті частини інтерфейсу, які змінились. Таким чином, знання класів, іменованих параметрів, колекцій, const, null safety та асинхронності – це базовий набір, необхідний для комфортної верстки інтерфейсу у Flutter.

Розмітка графічного інтерфейсу Flutter

Основою механізму розмітки у Flutter є віджети. У Flutter майже все є віджетом – навіть моделі розташування. Зображення, іконки й текст, які відображаються в застосунку, – це віджети. Але й те, що не відображається, теж віджети, наприклад, ряди, колонки чи сітки, які впорядковують, обмежують і

вирівнюють видимі елементи. Створення розмітки відбувається шляхом композиції віджетів, з яких будуються складніші структури.

У наступному прикладі (рис. 4) перший скріншот показує три іконки з підписами. На другому скріншоті додатково відображається візуальна структура рядків і колонок. Для цього було ввімкнено параметр *debugPaintSizeEnabled* = *true*, що дозволяє бачити рамки й допоміжну розмітку елементів інтерфейсу.



Рис. 4. Приклад іконок і візуального представлення структури їх віджетів.

На рис. 5 представлено дерево віджетів, яке відповідає попередньому прикладу.

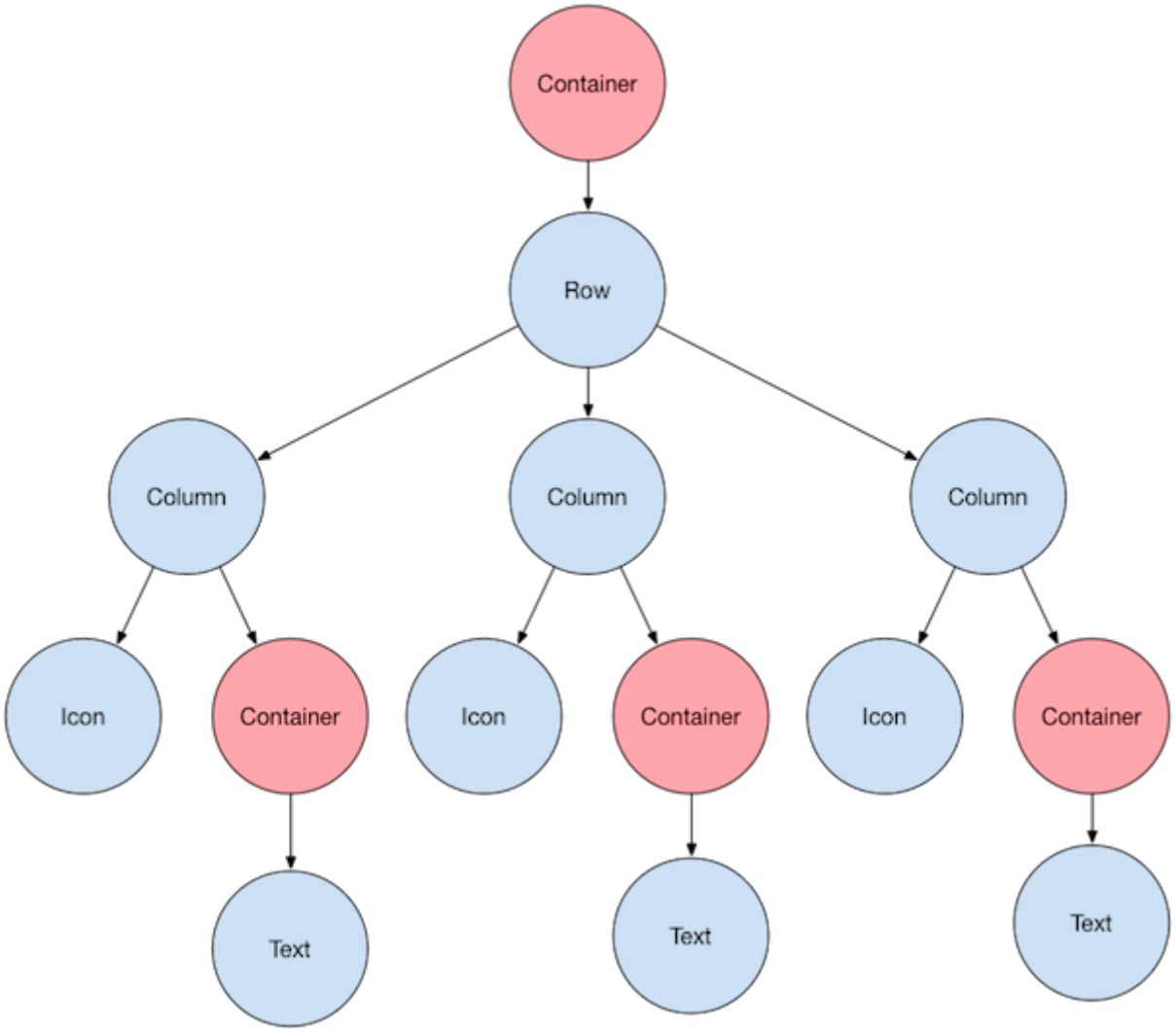


Рис. 5. Приклад дерева віджетів для рядка із трьох іконок.

Базові віджети Flutter

Flutter містить набір базових віджетів, серед яких найчастіше використовуються наступні: *Text, Row, Column, Stack, Container*.

Віджет *Text* дозволяє створювати текст із різними стилями та відображати його у вашому застосунку.

Row та *Column* – це так звані flex-віджети, які забезпечують побудову гнучких макетів у горизонтальному (*Row*) та вертикальному (*Column*) напрямках. Їхня концепція заснована на моделі розмітки flexbox у веб-розробці.

На відміну від лінійних макетів, *Stack* дозволяє розташовувати віджети один над одним у порядку малювання. Для управління позиціонуванням використовується віджет *Positioned*, який дозволяє задавати відстань від країв контейнера (*top, right, bottom, left*). Ця модель базується на принципах абсолютного позиціонування у вебі.

Віджет *Container* використовується для створення прямокутного візуального елемента. Його можна стилізувати за допомогою *BoxDecoration* (фон, рамка, тінь). Також *Container* підтримує відступи (*margin, padding*), обмеження розмірів та навіть тривимірні трансформації через матрицю.

Типи віджетів Flutter

Фреймворк Flutter вводить два основних класи віджетів: stateless та stateful.

Багато віджетів не мають змінного стану – їхні властивості не змінюються з часом. Прикладом можуть бути іконка чи текстова мітка. Такі віджети наслідують клас *StatelessWidget* і мають реалізовувати метод *build()*, який визначає інтерфейс.

Приклад користувацького stateless віджету, який відображає статичну колонку із картинкою (*String image*) і підписом (*String title*):

```
class OnboardingPageWidget extends StatelessWidget {
 final String image;
```

```
 final String title;

 const OnboardingPageWidget({
   super.key,
   required this.image,
   required this.title,
 });

 @override
 Widget build(BuildContext context) {
   return Column(
     mainAxisSize: MainAxisSize.min,
     children: [
       ConstrainedBox(
         constraints: const BoxConstraints(maxWidth: 375.0),
         child: Image.asset(image, fit: BoxFit.contain),
       ),
       const SizedBox(height: AppSpacing.medium),
       Padding(
         padding: const EdgeInsets.symmetric(horizontal: AppSpacing.mediumXx),
         child: Text(
           title.toUpperCase(),
           style: Theme.of(context).textTheme.headlineSmall,
           textAlign: TextAlign.center,
         ),
       ),
     ],
   );
 }
}
```

Однак, якщо поведінка чи вигляд віджета має змінюватися залежно від взаємодії користувача або інших факторів, він стає stateful. Наприклад, віджет-лічильник, що збільшує значення при натисканні кнопки. Такі віджети наслідують клас *StatefulWidget*. Сам віджет залишається незмінним, а змінний стан зберігається у відповідному класі, що наслідує клас *State*. Саме цей клас реалізує метод *build()*.

Щоразу, коли об’єкт стану змінюється (наприклад, інкрементується лічильник), потрібно викликати метод *setState()*, щоб повідомити фреймворк про необхідність повторного відтворення інтерфейсу.

Приклад користувацького віджета *DefaultSearchField* із збереженням стану кнопки для очищення поля введення, якщо воно не порожнє:

```
class DefaultSearchField extends StatefulWidget {
 final TextEditingController? controller;
 final Function(String)? onChanged;

 const DefaultSearchField({
```

```
    super.key,
    this.controller,
    this.onChanged,
  });

  @override
  State<DefaultSearchField> createState() => _DefaultSearchFieldState();
}

class _DefaultSearchFieldState extends State<DefaultSearchField> {
  late final TextEditingController _searchController;

  bool _showCloseButton = false;

  @override
  void initState() {
    super.initState();
    _searchController = widget.controller ?? TextEditingController();
  }

  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);
    return Padding(
      padding: const EdgeInsets.symmetric(
            horizontal: AppSpacing.normal,
            vertical: AppSpacing.small,
          ),
      child: SizedBox(
        height: 40.0,
        child: Theme(
          data: theme.copyWith(
            inputDecorationTheme: const InputDecorationTheme(),
          ),
          child: TextField(
            controller: _searchController,
            style: theme.textTheme.bodyMedium,
            decoration: InputDecoration(
              ...
              suffixIcon: _showCloseButton ? _buildCloseButton(context) : null,
            ),
            textInputAction: TextInputAction.search,
          ),
        ),
      ),
    );
  }

  Widget _buildCloseButton(BuildContext context) {
    return SizedBox.square(
      dimension: 40,
      child: InkWell(
        onTap: () => _searchController.clear(),
        child: Center(
          child: SvgPicture.asset(
```

```
            AppIcons.closeIcon,
            width: 20,
            height: 20,
          ),
        ),
      ),
    );
  }

  void _onChanged() {
    setState(() {
      _showCloseButton = _searchController.text.isNotEmpty;
    });
    widget.onChanged?.call(_searchController.text.trim());
  }
}
```

Розділення віджета і стану у Flutter дозволяє оновлювати інтерфейс, зберігаючи дані між перебудовами. *StatefulWidget* – це незмінна конфігурація: він описує віджет і містить параметри, передані батьківським віджетом. Об'єкт *State* зберігає змінні дані та керує їхнім життєвим циклом. Під час перебудови Flutter може створити новий об'єкт *StatefulWidget*, але зберегти наявний *State*, якщо віджет має відповідний тип і ключ у цьому місці дерева. Відокремлення стану від віджета дає змогу іншим віджетам працювати зі stateless і stateful однаково. Батьківський віджет може створювати новий екземпляр дочірнього у будь-який момент, не втрачаючи його стану. Фреймворк сам знаходить і повторно використовує існуючі об'єкти стану там, де це можливо.

Основи навігації у Flutter

Flutter підтримує навігацію між екранами та deep linking – відкриття певного екрана застосунку за посиланням, наприклад сторінки товару за адресою /products/42. Для простої навігації достатньо Navigator. Для складної маршрутизації, оброблення посилань і синхронізації з адресним рядком браузера доцільно використовувати підхід на основі Router, зазвичай через пакет маршрутизації, наприклад go_router. При цьому Router визначає конфігурацію маршрутів, а Navigator керує їх відображенням.

Більшість застосунків складаються з кількох екранів, які відображають різні типи інформації. Наприклад, один екран може показувати список товарів, а

після натискання на зображення товару відкривається інший екран із його детальною інформацією.

Екран застосунку у Flutter зазвичай реалізують як віджет. Для його відображення через Navigator створюють об’єкт маршруту (Route), наприклад MaterialPageRoute. Маршрут містить інформацію про побудову вмісту екрана та перехід до нього. В Android маршрут відповідає *Activity*. В iOS – *ViewController*.

Для навігації між маршрутами використовується *Navigator*. Віджет *Navigator* відображає екрани у вигляді стека, використовуючи правильні анімації переходів для цільової платформи.

Щоб перейти на новий екран, потрібно отримати доступ до *Navigator* через *BuildContext* та викликати методи, такі як *push()* або *pop()*. Navigator керує маршрутами за принципом стека: останній доданий маршрут відображається поверх попередніх. Метод push() додає новий маршрут, а pop() видаляє поточний верхній маршрут і повертає користувача до попереднього.

```
child: const Text('Open second screen'),
onPressed: () {
  Navigator.of(context).push(
    MaterialPageRoute<void>(
      builder: (context) => const SecondScreen(),
    ),
  );
},
```

Оскільки *Navigator* підтримує стек об’єктів *Route* (історію переходів), метод *push()* приймає об’єкт *Route*. Найчастіше використовується *MaterialPageRoute* – підклас *Route*, який визначає анімації переходів у стилі Material Design. Для додаткових прикладів можна звернутися до Flutter Cookbook або документації API Navigator.

Навігація між двома маршрутами має наступний вигляд:

1. Створити два маршрути;
2. Перейти на другий маршрут за допомогою *Navigator.push()*;
3. Повернутися до першого маршруту через *Navigator.pop()*.

Приклад застосунку із простою навігацією між двома екранами, де *FirstRoute* та *SecondRoute* – це звичайні віджети, що представляють окремі екрани застосунку.

```
void main() {
  runApp(const MaterialApp(title: 'Navigation Basics', home: FirstRoute()));
}

class FirstRoute extends StatelessWidget {
  const FirstRoute({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('First Route')),
      body: Center(
        child: ElevatedButton(
          child: const Text('Open route'),
          onPressed: () {
            Navigator.push(
              context,
              MaterialPageRoute<void>(
                builder: (context) => const SecondRoute(),
              ),
            );
          },
        ),
      ),
    );
  }
}

class SecondRoute extends StatelessWidget {
  const SecondRoute({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Second Route')),
      body: Center(
        child: ElevatedButton(
          onPressed: () {
            Navigator.pop(context);
          },
          child: const Text('Go back!'),
        ),
      ),
    );
  }
}
```

Робота з вкладками (tabs) є поширеним патерном у застосунках, що дотримуються принципів Material Design. У Flutter створення таб-інтерфейсу зручно реалізовано у складі бібліотеки material. Кроки:

1. Створити *TabController* для керування вкладками;
2. Описати самі вкладки (*Tab*);

3. Додати контент для кожної вкладки.

TabBar відображає панель вкладок, а TabBarView – вміст вибраної вкладки. Їхню роботу узгоджує TabController. У простих випадках використовують DefaultTabController, який автоматично створює контролер і надає його дочірнім віджетам. Перемикання вкладок у цьому прикладі змінює вміст поточного екрана й не додає маршрутів до стека Navigator.

Приклад застосунку із простою реалізацією перемикання між вкладками:

```
import 'package:flutter/material.dart';

void main() {
  runApp(const TabBarDemo());
}

class TabBarDemo extends StatelessWidget {
  const TabBarDemo({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: DefaultTabController(
        length: 3,
        child: Scaffold(
          appBar: AppBar(
            bottom: const TabBar(
              tabs: [
                Tab(icon: Icon(Icons.directions_car)),
                Tab(icon: Icon(Icons.directions_transit)),
                Tab(icon: Icon(Icons.directions_bike)),
              ],
            ),
            title: const Text('Tabs Demo'),
          ),
          body: const TabBarView(
            children: [
              Icon(Icons.directions_car),
              Icon(Icons.directions_transit),
              Icon(Icons.directions_bike),
            ],
          ),
        ),
      ),
    );
  }
}
```

Значення length у *DefaultTabController* має дорівнювати кількості вкладок у *TabBar.tabs* та кількості віджетів у *TabBarView.children*. Порядок вкладок має відповідати порядку їхнього вмісту.

Необхідні програмні засоби для виконання завдання

1. Android Studio IDE або Visual Studio Code.
2. Емулятор Android або симулятор iOS, якщо цільовими є мобільні платформи.
3. Xcode IDE, якщо цільовими є платформи Apple (iOS, macOs).
4. Chrome чи інший браузер, якщо планується підтримка Web.

Завдання

1. Створіть віддалений репозиторій на GitHub (або в іншій Git системі контролю версій). Використовуйте його для систематичного завантаження змін проекту у відповідності до посталених завдань цієї та наступних лабораторних робіт.
   1.1. У звіті вкажіть посилання на віддалений репозиторій.
   1.2. Якщо репозиторій приватний, надайте доступ лектору та керівнику ваших лабораторних робіт.
2. (Не обов'язково). Якщо під час розробки використовувалась LLM модель, то коротко опишіть використовувану LLM модель та harness (1 абзац), типовий робочий процес використання попередньо сформульованих вимог та дизайну для проектування та кодування застосунку (1 абзац) – надайте приклад одного запиту до LLM моделі.
3. Зробіть частину екранів застосунку **без реалізації бізнес-логіки** у відповідності до розробленого дизайну застосунку в лабораторній роботі №2 та подайте у звіт скриншоти розроблених екранів:
   3.1. Реєстрація та авторизація користувачів із можливістю відновлення пароля через електронну пошту;
   3.2. Керування профілем користувача із можливістю заміни пароля;
   3.3. Головний екран (сторінка) застосунку із швидким доступом до основних функцій та даних.
4. "Без реалізації бізнес-логіки" означає, що не потрібно виконувати інтеграцію із Firebase, REST API сторонніх сервісів чи сторонніх

бібліотек. В звіті надайте приклад коду для екрану авторизації та домашньої сторінки.

4.1. Приклад №1. На екрані логіну користувача кнопка “Увійти” може просто відкривати наступний екран домашньої сторінки, як у випадку успішної авторизації – не обов’язково реалізовувати валідацію полів даних та виконання реального запиту до бекенду.

4.2. Приклад №2. Замість реальних даних користувача із віддаленого сервера можна використати hardcoded-дані: статична змінна із іменем користувача *static const String _userName = ‘John Doe’* для відображення імені на головній сторінці застосунку.

5. Віджети, які будуть повторно використовуватись у різних місцях програми, потрібно зберігати в окремому пакеті (наприклад, *core/widgets*). Наприклад, стилізоване текстове поле введення, стилізовані кнопки підтвердження чи скасування дій, карточки для різних типів користувацьких даних, які будуть відображені в списках, тощо. В звіті надайте приклад одного чи двох віджетів.
6. Відобразити у звіті скриншот структури пакетів проекту та їх вміст після виконання завдання лабораторної роботи.

Вимоги до звіту

Захист лабораторних робіт включає також виконання звіту у текстовому редакторі. Рекомендації до форматування звіту: розмір шрифту основного тексту – 14 пт; міжрядковий інтервал – 1,25; абзацний відступ зліва – 1 см; перед- й післяабзацні міжрядкові інтервали – 0; вирівнювання основного тексту – двостороннє; заголовки відцентровані з накресленням – Bold, усі поля сторінки – 2 см, орієнтація аркушів книжкова. Аркуші звіту формату А4 нумеруються без вказання номера на першій сторінці.

Складовими звіту є титульний аркуш, тема, мета, постановка завдання, отримані результати, висновки.

У розділі «Отримані результати» розмістити відповіді на питання завдання.

У висновках зазначити й охарактеризувати отримані результати в ході виконання лабораторної роботи.

Контрольні питання для самоперевірки

1. Чим відрізняється реактивний декларативний підхід у Flutter від традиційних UI-фреймворків?
2. Яка проблема виникає при ручному оновленні інтерфейсу у складних застосунках?
3. У чому полягає основна ідея React-подібної моделі у Flutter?
4. Яке призначення методу *build()* і чому він повинен бути без побічних ефектів?
5. Що таке “побічні ефекти” для методу *build()*?
6. Які особливості мови Dart роблять її придатною для створення UI?
7. Що таке const-конструктор і яку оптимізацію він забезпечує у Flutter?
8. Яке призначення дерева віджетів у Flutter?
9. Яке призначення дерева елементів у Flutter?
10. Яке призначення дерева рендер об’єктів у Flutter?
11. Які три рівні (дерева) використовуються для побудови UI у Flutter?
6. Що таке віджет у Flutter і чим StatelessWidget відрізняється від StatefulWidget?
7. З яких компонент складається StatefulWidget?
8. Як реалізовано управління і зміну стану через StatefulWidget?
9. Для чого використовується метод *setState()*?
10. Чому розділення віджета і стану є важливим у Flutter?
11. Чим іменовані параметри у Dart зручні під час створення віджетів?
12. Як у Flutter можна використовувати колекції List та Map для побудови інтерфейсу?
13. Що таке null safety і як воно допомагає уникати помилок у UI-коді?

14. Яке призначення мають віджети Row та Column?
15. Чим Stack відрізняється від Row і Column у розмітці інтерфейсу?
16. Які можливості надає Container для стилізації інтерфейсу?
17. Для чого використовується параметр `debugPaintSizeEnabled` під час розробки?
18. Чому у Flutter майже все вважається віджетом, включно з елементами, які не відображаються напряму?
19. Як Navigator керує стеком маршрутів?
20. Що відбувається під час викликів push() та pop()?
21. Яке призначення MaterialPageRoute під час переходу між екранами?
22. Що таке deep linking?
23. Для яких застосунків доцільно використовувати навігацію на основі Router?
24. Як взаємодіють DefaultTabController, TabBar і TabBarView під час роботи з вкладками?

Список рекомендованої літератури

1. Недашківський О. Л. Кросплатформна розробка мобільних застосунків. Навчальний посібник. / О. Л. Недашківський, І. І. Гусєва; КПІ ім. Ігоря Сікорського, Київ, 2023. – 221 с.
2. Richard Rose: Flutter and Dart Cookbook: Developing Full-Stack Applications for the Cloud 1st Edition / O'Reilly Media – 2023. – 308 p.
3. UI | Flutter [Електронний ресурс]. - Режим доступу: https://docs.flutter.dev/ui
4. Introduction to Dart [Електронний ресурс]. - Режим доступу: https://dart.dev/language
5. Layout | Flutter [Електронний ресурс]. - Режим доступу: https://docs.flutter.dev/ui/layout

6. Layout tutorial | Flutter [Електронний ресурс]. - Режим доступу: https://docs.flutter.dev/ui/layout/tutorial
7. Navigation and routing | Flutter [Електронний ресурс]. - Режим доступу: https://docs.flutter.dev/ui/navigation
8. Navigate to a new screen and back [Електронний ресурс]. - Режим доступу: https://docs.flutter.dev/cookbook/navigation/navigation-basics